

Cross Validation

The Problem and why CV needed:

The core Problem:- When we train a model, we want to answer one question $\rightarrow$ How well will this model perform on unseen (new data)?

$\rightarrow$ But we don't have future data, so we simulate it using the available dataset.

• Naive approach:- (The Problem)

We usually Train on training data
Evaluate on test data, But this causes 2 major problems

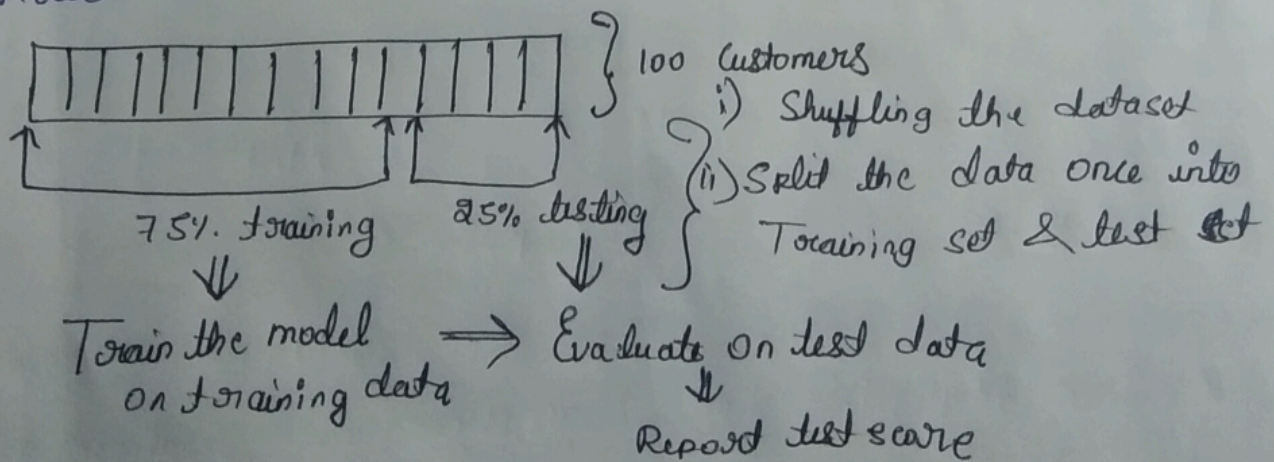
Problem 1:- if the dataset is small
model performance depends heavily on which data went into train vs test
• for diff split $\rightarrow$ diff accuracy

Problem 2:- Overfitting to test data set

- If we Try many models, Tune parameters, Re-check test score repeatedly. Then
- model indirectly learns test data, This causes data leakage.

The Hold-Out Approach:- [Train-test-split] [Baseline method]

$\rightarrow$ Hold out means:- The simplest Evaluation method.

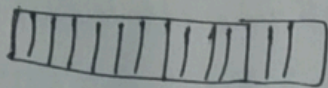


Problems with Hold-Out Approach:-

i). High Variability (Variance):-

→ Result depends on a single split.
one split $\neq$ representative of whole data

- The Performance of the model can be very sensitive to how the data is divided into training and testing sets.
- If the split is unfortunate, the training set may not be representative of the overall distribution of data.
- or the test set might contain unusually easy or difficult examples. This leads to high variance in the estimation.



i) shuffle ii) Train-test-split

iii) random-state = 1, 2, 3
accuracy = 0.72, 0.76, 0.77

which model need to be deployed

ii) Data Inefficiency (waste of data):-

- Test set data is never used for training,
- when data is limited, this hinders learning; model trains on less information.

→ Cause high bias :- Bias = error due to underfitting.

- when model doesn't learn patterns well, and because of inefficient data or too simple model will cause high bias.

iii) Not reliable for model comparison:-

- Two models may perform differently just because of the split
- Hard to say which model is truly better (misleading)

iv) Risk of overfitting to test set:- (due to hyperparameter tuning)

- Repeatedly checking test score
- Tuning model based on it
- so test data becomes indirectly part of training (causes data leakage)

Why hold out approach used then:-

i) Simple and Fast:- only one split, only one training, very low computation cost

ii) works well for large dataset:-

- when dataset is very large: Train-test is still huge
- losing some data does not increase bias much.
- Performance estimate becomes stable.

(Problems of hold-out reduce as data size increases)

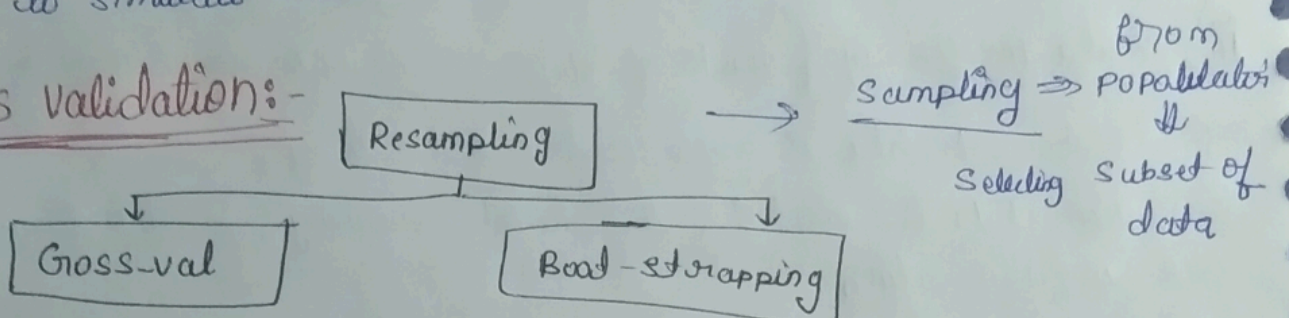
iii) Used as a final set (Test):-

→ Gross-validation is used for model selection

→ Hold out test set is kept untouched

why:- to get an unbiased final evaluation
to simulate real-world unseen data.

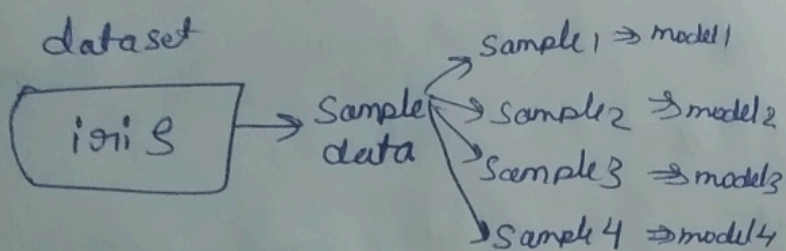
Gross validation:-



Sampling:- Selecting a subset of data from a larger population or dataset

- Samples represent the population Ex:- iris dataset $\Rightarrow$ sample data not world iris data

Resampling:-



Resampling the process of ~~knowing~~ repeatedly drawing samples from a dataset to estimate model performance more reliably.

why reSampling:-

It reduces dependence on a single data split and provides a stable performance estimate.

diff b/w resampling and standard
Random split is done Once, resampling used multiple splits

"Cross Validation is a resampling technique used to evaluate a model by training and testing it multiple times on different splits of the same data."

Why need :- to solve all the problems of (hold-out)

How cross validation works:-

- Splits the data into k Equal parts (folds)
- for Each iteration: use k
 - use $k-1$ fold for model training
 - use 1 fold for testing (validation)
- Repeat until Each fold becomes validation once.
- Average the results to estimate model's overall performance

Ex:- 1000 Dataset

↓
Training set (900)

↓
Apply cross validation

↓
Assume $k\text{-fold} = 5$

↓
Each fold = $900/5 = 180$ Sample

↓
 $k\text{-fold}$ training and validation

• Training $\Rightarrow 4$ fold $\Rightarrow 720$ Samples

• Validation $\Rightarrow 1$ fold $\Rightarrow 180$ Samples

Repeat until Each fold become validation Once

↓
You get 5 validation scores, Take the average

↓
Average is used for model comparison and Hyperparameter tuning

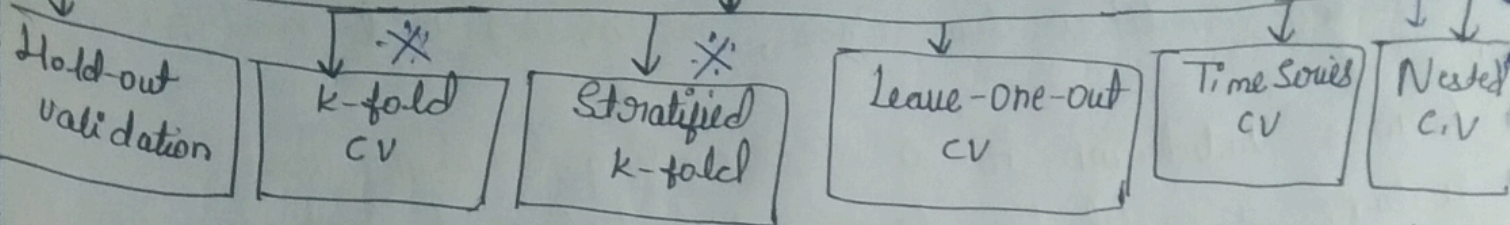
test-test (100) samples

↓
(never touched during model selection)

↓
after choosing best model, train 1 model on 900 samples

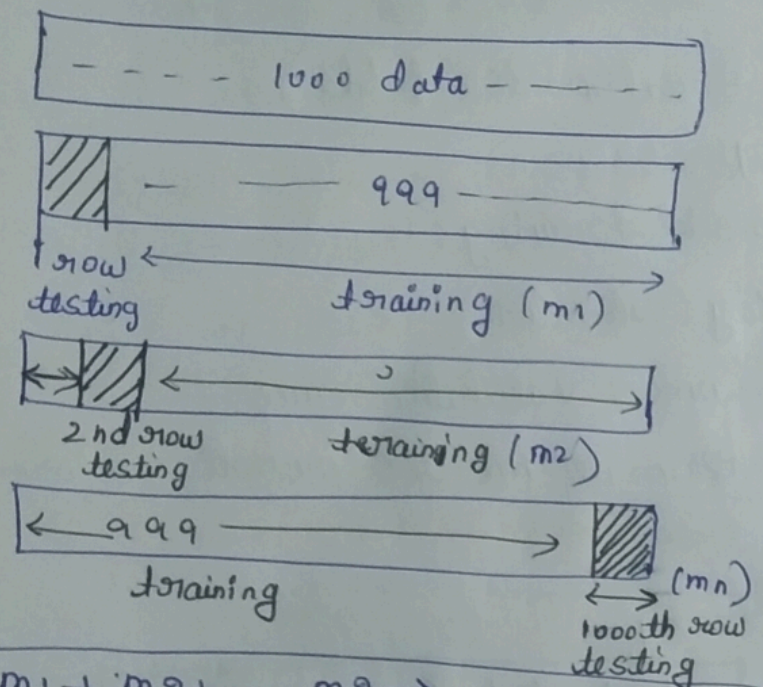
↓
Evaluate the model on the 100 unseen test samples

Types of Cross Validation



Leave-One-Out Cross validation: - (R^2 -score will not work) ^{blows} ^{1 test data}

LOOCV is a Cross Validation method, where each data point is used once as validation and the rest as training.



$m_1 + m_2 + \dots + m_n \Rightarrow \text{avg} \Rightarrow \text{final accuracy score}$

Pros:-

- uses almost all data for training
- very low bias
- useful when dataset is very small,

When to use:-

- small dataset
- Balanced datasets
- when need for less biased performance

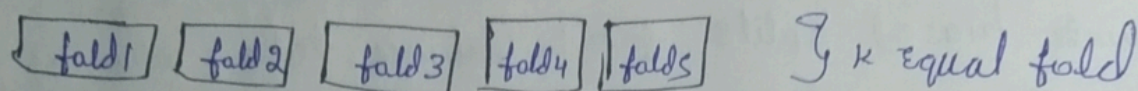
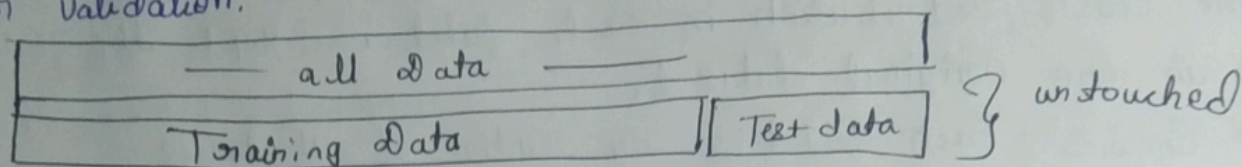
Some model give $\rightarrow$ high bias } in that scenario
~~low~~ low variance } LOOCV can be used, but variance will increase

Problems with LOOCV

- High Variance ^(accuracy is varying) ^{accuracy on}
 → validation set has only 1 point
 → performance estimate fluctuates a lot (e.g., 86, 77, 95)
- Very Expensive:-
 → Training model N times
 → Not scalable
- Not used much in industry
 → Too slow
 → unstable estimate
- R^2 -score not used with LOOCV, since validation set has only 1 sample
- Not ideal for imbalanced data

K-fold Cross Validation

K-fold Cross Validation is a resampling method where the dataset is split into K equal folds, and the model is trained K times, each time using K-1 folds for training and 1 fold for validation.



Split	fold 1	2	3	4	5
1	1	fold 2	3	4	5
2	1	2	fold 3	4	5
3	1	2	3	fold 4	5
4	1	2	3	4	fold 5
5	1	2	3	4	5

⇒ avg result

Advantages of K-fold

- i) Reduction of Variation:- By averaging over K different random partitions, the variance of the performance estimate is less sensitive to the particular random split of the data.
- ii) Computationally Inexpensive:- Take less time & space compared to cross validation.
- iii) Efficient use of data , iv) Helps in model selection & hyperparameter tuning.

Disadvantage:-

- i) High bias:- if K is too small, there could be high bias, if the test set is not representative of the overall population.
- ii) Not work with imbalanced data:- There is risk in partitioning, some fold might not contain any samples of minority class, which lead to performance metric.

100 rows ⇒	95% yes	5 no	K fold = 5	3 20	7 20	7 20	7 20	7 20	avg accuracy ⇒ 97%
			accuracy ⇒	100	100	100	100	100	

when to use:- when you have sufficiently large dataset.

when not to use:- when data is not evenly distributed (imbalanced data)

Stratified K-fold C.V:-

Stratified K-fold is a variation of K-fold CV where the class distribution in each fold is kept the same as in the original dataset.

→ Ensures Each fold is representative of all classes

→ Prevents biased splits for imbalanced data, why means

Problem with normal K-fold:-

Classification

Yes (95%)

No (5%)

Ex

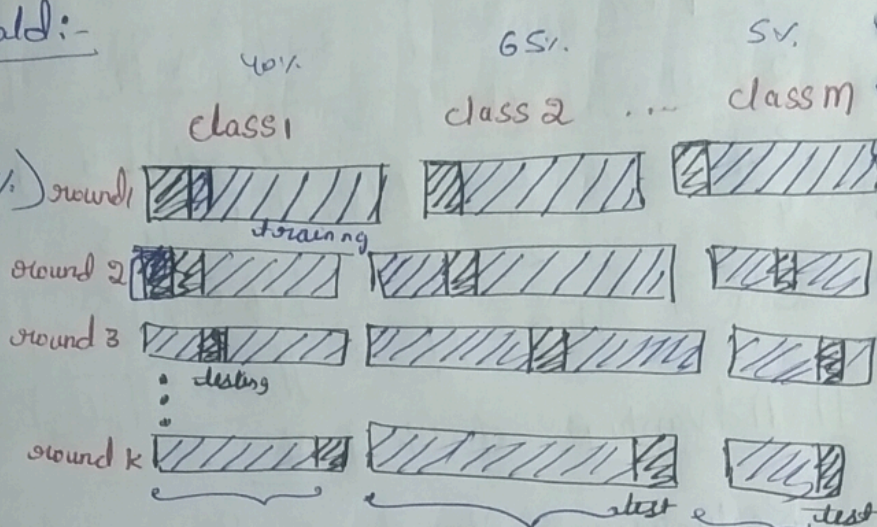
class 1	class 2	class 3
40%	55%	5%

dataset 100 samples

class 0 : 80 , class 1 : 20

$K=5$, $\Rightarrow 100/5 = 20$ (class 0: 16, class 1: 4)

Each fold keeps 16 class 0, 4 class 1



training but contains (same class distribution in each round)

(not for regression only K-fold or stratified on binned target.)

Then:- train on $K-1$ fold, validate on fold

advantages:-

i) Preserves class distribution $\Rightarrow$ Each fold is representative of whole data.

ii) Reduced bias:- validation performance is reliable

iii) Better for Imbalanced data $\Rightarrow$ model learns minority class in every fold,

use cases in industry

• fraud detection, medical diagnosis, Rare Event classification

Time Series Cross Validation

Time Series CV Evaluates model without breaking time order, training on past data and testing on future data

why normal k-fold fails here:-

- k-folds shuffles data
- future data may appear in training
- Causes data leakage.
- Leads to over-optimistic performance

how Time-Series CV works:-

fold	Train	Test
1	[1 - 100]	[101 - 120]
2	[1 - 120]	[121 - 140]
3	[1 - 140]	[141 - 160]

} • Training window moves forward
• Test data is always future

when to use:-

- stock price , • Sales forecasting • Sensor / log data
- any data with timestamps.
- Time Series CV preserves temporal order and prevents future data leakage.

Nested Cross Validation:-

Nested cross validation uses two loops:

- inner loop ⇒ hyperparameter tuning
- outer loop ⇒ model evaluation

Why Nested CV needed

→ Problem

- Tuning hyperparameter on test data
- Then reporting performance on same data

→ leads to optimistic bias / data leakage

Solution:-

- Nested CV separates tuning & Evaluation

How it works

- outer CV splits data ⇒ test fold
- inner CV tunes hyperparameters on training folds
- best model evaluated on outer test fold.
- ⇒ final score = an avg of outer folds